Strategy Research and Trading SOP

Version: 2026-10-16. Platform: Windows PowerShell with Python 3.10; Linux differences are noted below. Read this manual from the implementation repository root. The current platform supports research, validation, persistent paper execution and gated Kraken spot execution. It does not authorise a real order. Historical and accelerated demonstrations are software tests, never prospective profitability evidence.

1. Install and verify

The implementation repository contains licensed TWSQ code and must remain private. The independent research publication repository excludes that code. Clone only with authorised access:

```
git clone https://github.com/avyas1810/kraken-quant.git
Set-Location kraken-quant
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements-ci.txt -r requirements-research.txt
python -m pytest -q tests
python -m research.phase2.verify_results --results phase2_results/reviewed_20261016
```

Select Python 3.10 when creating the environment. Run modules from the repository root; no editable installation of the legacy setup.py is necessary. If PowerShell prevents environment activation, invoke .venv/Scripts/python.exe directly. Linux uses python3.10 -m venv .venv and source .venv/bin/activate; the subsequent module commands are identical. Do not globally change execution policy to run this project.

Expected verification: all tests pass; artifact verification reports status=verified, tables=9, bootstrap_replications=1999 and placebo_trials_per_method=999. Dependency installation needs package-network access. No API key is needed for tests or public market data.

The five daily Kraken USD assets and chronological boundaries come from examples/research_config.py: BTC, ETH, XRP, SOL and ADA; training through 2023, validation in 2024, locked test in 2025 through 2026-09-30. Never edit that configuration to turn the inspected nine-asset Binance study into a new holdout. The backtest CLI rejects test requests before reading data.

2. Create an isolated operator run

Use a new run directory for create-once exports and experiments. Do not delete a database or frozen artifact to bypass a conflict.

```
$taskRun = ".shadow-runtime/operator-" + (Get-Date -Format "yyyyMMdd-HHmmss")
New-Item -ItemType Directory -Path $taskRun | Out-Null
```

All examples below use this variable. On Linux, assign taskRun=.shadow-runtime/operator-$(date +%Y%m%d-%H%M%S), create it with mkdir, and use $taskRun in paths. Runtime databases, exports, caches and .env remain ignored by Git.

3. Define a custom strategy

A strategy implements target_weights(context). It receives copies of completed historical daily bars, current quantities/cash/equity and optional causal features. Return instrument-to-weight mappings; omitted instruments target zero. Return None to hold quantities. Spot weights must be finite, nonnegative, with total at most one. Strategy code must contain no exchange submission or credential handling.

The working module examples/custom_strategy_example.py contains CustomMomentum, backtest_factory(parameters, seed), paper_factory() and paper_demo_factory(). Backtest factories return a strategy; paper factories return StrategyPipeline. The demo paper factory uses three-day momentum solely to exercise short software fixtures. Its default five-percent BTC allocation is separate from the frozen empirical candidate.

```
from kraken_quant.strategy import StrategyPipeline

class MyStrategy:
    def target_weights(self, context):
        weights = {}
        for key, bars in context.historical_data.items():
            ready = len(bars) > 20
            rising = ready and bars.close.iloc[-1] > bars.close.iloc[-21]
            weights[key] = 0.05 if key.base == "BTC" and rising else 0.0
        return weights

def backtest_factory(parameters, seed):
    return MyStrategy()

def paper_factory():
    return StrategyPipeline(MyStrategy())
```

Save custom code in an importable trusted module. External data/model files need input hashes and training-only fit provenance. Arbitrary Python is not sandboxed. Indicators and feature objects may be supplied with --feature-factory module:function; that factory receives parameters/seed and returns the completed-history feature builder. Future or unfinished bars are rejected by the pipeline.

4. Run the complete backtest examples

```
python -m examples.run_btc_momentum --demo --output "$taskRun/btc-example"
python -m examples.run_multi_momentum --demo --output "$taskRun/multi-example"
[ordered]@{lookback=20;weight=0.05} | ConvertTo-Json | Set-Content -Encoding utf8
"$taskRun/custom-parameters.json"
python -m kraken_quant.platform_cli backtest --demo --factory
examples.custom_strategy_example:backtest_factory --parameters-file "$taskRun/custom-
parameters.json" --fee 0.004 --spread 0.001 --slippage 0.0005 --output "$taskRun/custom-
backtest"
```

These commands use generated fixed-seed prices and explicitly report SYNTHETIC SOFTWARE DEMO; NOT EMPIRICAL EVIDENCE. They execute signals, next-open fills, funded accounting, fees/spread/slippage and a buy/hold benchmark through the actual repository engine. Both built-in examples produce fills. The custom factory command uses all configured histories while allocating to BTC; its benchmark is the equal-weight selected-universe benchmark.

Parameter files avoid PowerShell 5.1 native JSON quoting problems. Inline --parameters and --parameters-file cannot be used together. Output directories must be new. Do not interpret synthetic Sharpe as research performance.

5. Load authentic historical data

CSV files must be named BTC_USD.csv, ETH_USD.csv, XRP_USD.csv, SOL_USD.csv and ADA_USD.csv. Each has timestamp,open,high,low,close,volume columns; timestamps identify timezone-aware UTC daily opens. Supply complete selected windows. Duplicate/missing days, nonfinite values, invalid prices and OHLC relationships fail; no forward filling is performed.

Set the operator's real data location, then run the implemented interface:

```
$taskData = "C:/Data/authorised-daily-bars"
python -m kraken_quant.platform_cli backtest --data-dir $taskData --venue kraken
--strategy btc_momentum --split train --start 2022-01-01 --end 2022-12-31 --output
"$taskRun/real-btc"
python -m kraken_quant.platform_cli backtest --data-dir $taskData --venue kraken
--strategy multi_momentum --split validation --start 2024-01-01 --end 2024-12-31
--output "$taskRun/real-multi"
```

The path above is an operator-supplied dependency, not a bundled dataset. These command forms are exercised with validated CSV fixtures in tests. Exact real outcomes require authentic data. Use --venue binance only for genuine Binance inputs; USDT remains a different quote currency and is labelled as such. A CSV filename does not prove source provenance. Record original-file hashes and source metadata independently.

6. Inspect accounting and performance

Each backtest exports equity.csv, positions.csv, trades.csv, metrics.json and manifest.json. Equity includes cash, daily PnL/returns, benchmark wealth and financing. Trades include decision/fill times, quantities, reference/fill prices, fees, spread and slippage. Metrics include net return, volatility, Sharpe, Sortino, drawdown, turnover, executed-fill count and costs. Fill count is not completed round-trip count.

```
Get-Content "$taskRun/custom-backtest/metrics.json"
Import-Csv "$taskRun/custom-backtest/trades.csv" | Select-Object -First 5
Import-Csv "$taskRun/custom-backtest/equity.csv" | Select-Object -Last 5
Get-Content "$taskRun/custom-backtest/manifest.json"
```

Manifests record source snapshots, factory/feature source hashes, Git revision, parameters, costs, seed and output hashes. Undefined ratios are null, not infinite alpha. Backtests model full fills; separate execution stress explores liquidity/impact/delay, but is not an exchange queue simulator.

7. Statistical validation and independent review

```
python -m research.extended.validation --equity "$taskRun/custom-backtest/equity.csv"
--trial-dispersion 0.05 --replications 199 --output "$taskRun/custom-validation.json"
python -m research.phase2.verify_results --results phase2_results/reviewed_20261016
```

The first command is a fast software demonstration. The dispersion 0.05 is an explicit illustrative assumption; it is not an estimated historical trial count or live confidence claim. For research, use justified trial dispersion and the default 1999 replications. Daily validation reports block-bootstrap absolute/paired Sharpe, mean-excess uncertainty, DSR sensitivity and benchmark HAC. Its circular alignment placebo tests correlation alignment and is explicitly not the frozen regime-filter profitability placebo. Daily equity must be complete, positive, finite and UTC; missing data is rejected.

For the recovered original study, these implemented commands require the original local hash-verified corpus:

```
python -m research.extended.audit --root C:/Users/avyas --output "$taskRun/original-
audit"
python -m research.extended.trials --returns
C:/Users/avyas/phase1_results/4h_net_returns.csv --output "$taskRun/trial-audit"
python -m research.phase2.run_validation --original-research-root C:/Users/avyas
--output "$taskRun/original-phase2"
```

The Phase2 runner reproduces the regime-mask placebo, corrected DSR, paired bootstrap, BTC HAC and stability using the frozen accounting. It downloads no substitute data. Bootstrap uses 12/36/72/144 four-hour bars and 1999 draws per length; placebo uses 999 trials per method. Existing inspected OOS is not pristine. See docs/research_audit.md for two automated review roles, known 18 correlated trials, unknown experiments, execution assumptions and survivorship limitations.

8. Register, freeze and inspect qualification

For a copy-ready tutorial using the executed custom backtest:

```
python -m examples.freeze_operator_demo --manifest "$taskRun/custom-
backtest/manifest.json" --output "$taskRun/tutorial-candidate.json"
```

This verifies the existing backtest outputs, seals the custom source/parameters and sets a future UTC prospective start. It is explicitly research_only and can never qualify for real trading. A genuine candidate uses an operator-reviewed complete specification:

```
python -m kraken_quant.platform_cli freeze --spec candidate_spec.json --output
"$taskRun/candidate.json"
```

Supply actual strategy factory/parameters, causal features, costs, immutable source/data hashes, code identity, configured universe, risk limits and a future prospective_start. Freeze refuses overwrite or retrospective test release. Keep every prior version. Do not replace a frozen source hash after changing code; register a new version and retain the old record.

Qualification preserves Sharpe >=0.5, drawdown <=15%, positive net/benchmark-relative returns, at least 3 positive assets, 2 positive neighbouring parameters, 30 completed round trips, nonnegative doubled-spread/slippage return, positive 95% bootstrap mean-excess lower bound and independent review. Operational requirements additionally include 30 actual consecutive daily decisions over at least 29 days, 20 simulated order lifecycles, >=99.5% completion, no missing days/faults/heartbeat gaps and verified fees/event times. Missing evidence fails closed. No software demonstration authorises live trading.

9. Persistent accelerated paper replay

```
python -m examples.create_operator_fixtures --output "$taskRun/paper-fixtures"
python -m kraken_quant.paper_cli replay --bars "BTC/USD=$taskRun/paper-
fixtures/BTC_USD.csv" --database "$taskRun/paper.sqlite" --strategy
examples.custom_strategy_example:paper_demo_factory --strategy-id custom-
momentum-3-software-replay-v1
python -m kraken_quant.paper_cli export --database "$taskRun/paper.sqlite" --output
"$taskRun/paper-export.json"
python -m kraken_quant.paper_cli inspect --database "$taskRun/paper.sqlite"
```

The fixture is explicitly synthetic post-lock daily data. Expected: 14 accelerated decisions, 4 durable fills, cash about 99.71498, no remaining BTC and marked PnL about -0.28502 at 0.4% scenario fees. Evidence reports zero real/prospective decisions. The exact ledger is reconstructed with Decimal accounting. Re-running the same replay resumes without duplicate fills. Replay dates and fast

execution never count as real elapsed observations.

For authentic post-lock replay, replace the fixture with genuine CSVs and retain source hashes. Add repeated --bars SYMBOL=FILE arguments for multi-asset common-boundary histories. Use --candidate PATH to bind an exactly frozen candidate instead of a discretionary --strategy. A changed engine/candidate/configuration requires a new database, preserving the old evidence.

10. Genuine public Kraken shadow collection

```
python -m kraken_quant.paper_cli diagnose --symbol BTC/USD
python -m kraken_quant.paper_cli once --database "$taskRun/public-flat.sqlite" --start
2026-10-01T00:00:00Z --strategy-id flat-safety-v1 --symbols BTC/USD ETH/USD XRP/USD
SOL/USD ADA/USD
python -m kraken_quant.paper_cli evidence --database "$taskRun/public-flat.sqlite"
```

Expected: authentic recent exchange event timestamp, bid < ask, one processed completed daily decision and no real submission. Flat observations prove feed/accounting operation, not strategy alpha or twenty trading lifecycles. Prices/times vary. The multiplex WebSocket v2 BBO reader retains exchange timestamps, monotone event-time validation, bounded reconnect/read limits and crossed/stale rejection. The channel has no documented sequence number; packet completeness is not falsely claimed.

After a real candidate's frozen prospective start, supervised collection is available:

```
python -m kraken_quant.paper_cli collect --candidate "$taskRun/candidate.json"
--database "$taskRun/prospective.sqlite" --start 2026-10-01T00:00:00Z --duration-days 30
```

Run in a supervised foreground console, keep the machine awake and configure a separate health process. --candidate binds parameters, costs, source hashes and identity; do not combine with --strategy. At least 30 real daily decisions and 20 actual paper lifecycles require future observations. This manual does not leave a collector running. Engine changes invalidate restart identity and require a new evidence experiment.

Export durable qualification evidence after observations exist:

```
python -m kraken_quant.paper_cli export --database "$taskRun/prospective.sqlite"
--candidate "$taskRun/candidate.json" --qualification-evidence --output
"$taskRun/prospective-evidence.json"
python -m research.extended.validation --equity "$taskRun/prospective-
evidence.equity.csv" --trial-dispersion 0.05 --output "$taskRun/prospective-
statistics.json"
python -m kraken_quant.paper_cli export --database "$taskRun/prospective.sqlite"
--candidate "$taskRun/candidate.json" --qualification-evidence --validation-report
"$taskRun/prospective-statistics.json" --robustness-report "$taskRun/reviewed-
robustness.json" --review-report "$taskRun/independent-review.json" --output
"$taskRun/reviewed-evidence.json"
python -m kraken_quant.platform_cli qualify --candidate "$taskRun/candidate.json"
--evidence "$taskRun/reviewed-evidence.json" --output "$taskRun/qualification.json"
```

Do not use the illustrative dispersion for real qualification without justification. Statistics must match the exported equity-file hash. Review/robustness reports must bind candidate and equity hashes; they represent actual reviewed measurements, not booleans to invent. Missing review/statistics/stress/cross-asset/parameter evidence remains explicitly unqualified. Export paths must be new; repeated exports preserve old evidence. Genuine input collection and review are external prerequisites, not missing CLI functionality.

11. Monitor, stop, recover and restore paper state

```
python -m kraken_quant.paper_cli health --database "$taskRun/public-flat.sqlite"
python -m kraken_quant.paper_cli halt --database "$taskRun/public-flat.sqlite" --reason
"operator emergency drill"
python -m kraken_quant.paper_cli recover --database "$taskRun/public-flat.sqlite"
--reason "paper-only cycle and ledger reviewed"
python -m kraken_quant.paper_cli resume --database "$taskRun/public-flat.sqlite"
--reason "inputs and ledger verified"
python -m kraken_quant.paper_cli backup --database "$taskRun/public-flat.sqlite"
--output "$taskRun/paper-backup.sqlite"
python -m kraken_quant.paper_cli restore --database "$taskRun/paper-backup.sqlite"
--output "$taskRun/restored-paper.sqlite"
```

Health returns exit 1 for a stopped collector, expired heartbeat, halted or corrupt state; this is expected, not a reason to suppress checks. The kill is persistent. Recovery reconstructs committed fills; an uncertain paper cycle is abandoned, never replayed. Restore writes a new file and leaves it halted. Never operate original/restored instances concurrently as the same prospective experiment. Inspect and reconcile before manual resume.

For an operator-configured HTTPS alert recipient, store its URL in KRAKEN_HEALTH_WEBHOOK and deliberately enable delivery:

```
python -m kraken_quant.paper_cli health --database "$taskRun/public-flat.sqlite"
--alert-url-env KRAKEN_HEALTH_WEBHOOK --send-alert
```

Only safe health booleans are sent. The recipient is not supplied by this project and no alert was sent during development. Schedule this health command independently of collection, using Windows Task Scheduler or Linux systemd/cron, so it can notice a dead process. Do not put credentials or private account state in task arguments/logs.

12. Verify account permissions, fees and risk

A dedicated read-only .env needs KRAKEN_READONLY_API_KEY and KRAKEN_READONLY_API_SECRET. A separate trading key uses KRAKEN_TRADING_API_KEY and KRAKEN_TRADING_API_SECRET. Never print the file. Required read-only privileges are query-funds, query-open-trades and query-closed-trades; optional query-ledger is allowed. Funding/trading/unknown privileges are rejected by the observer. The trading transport requires its exact documented read/order/cancel allowlist, with no deposit/withdrawal or funding access.

With actual dedicated read-only credentials, accept a quiet account baseline and collect verified fees:

```
python -m kraken_quant.paper_cli pin-exchange --database "$taskRun/public-flat.sqlite"
--env-file .env --start 2026-10-01T00:00:00Z --reason "dedicated read-only account
reviewed"
python -m kraken_quant.paper_cli once --database "$taskRun/public-flat.sqlite" --env-
file .env --start 2026-10-01T00:00:00Z --strategy-id flat-safety-v1 --symbols BTC/USD
ETH/USD XRP/USD SOL/USD ADA/USD
```

Do not repin to hide discrepancies. The real-account mirror remains separate from paper balances. Unexpected orders/trades/transfers halt. A key fingerprint is API-key identity, not legal account identity; verify ownership/dedicated-account status in Kraken itself. Real verification is unavailable when the dedicated credentials are absent.

Hard spot caps remain $20/order, $40/asset, $100 total exposure and $2 daily loss, with pending commitments. Precision/minimums/status and account-specific fees come from Kraken. No

leverage/borrowing is permitted. Exchange-minimum conflicts fail rather than scaling caps. Record genuine midnight equity for live daily-loss baselines; never reset a loss by restarting.

13. Test live execution safely, then separately authorise

```
python -m kraken_quant.live_cli simulate --database "$taskRun/live-simulation.sqlite"
python -m kraken_quant.live_cli inspect --database "$taskRun/live-simulation.sqlite"
python -m kraken_quant.live_cli kill --database "$taskRun/live-simulation.sqlite"
```

Expected: SIMULATION_ONLY, real_order_submissions=0, two simulated submissions, partial-cancel and lost-acknowledgement reconciliation, two deduplicated fills, USD cash 984.94000 and BTC 0.15. Restart preserves orders/reservations. Default-disabled and stale/funds/loss/exposure/kill scenarios are covered by tests. The signed in-memory HTTP simulation verifies HMAC-style transport without contacting Kraken.

Real read-only account observation through the live transport is implemented:

```
python -m kraken_quant.live_cli account-check --database "$taskRun/live.sqlite" --env-
file .env
python -m kraken_quant.live_cli reconcile --database "$taskRun/live.sqlite" --env-file
.env
```

These require actual dedicated credentials. No order is submitted. Initial reconciliation requires a quiet dedicated account. Unknown external activity, unsupported assets, inconsistent history or balance discrepancies halt.

Real mutation requires a separate operator-authored approval, a qualified frozen candidate and verified prospective/qualification artifact hashes, active for at most 24 hours. The completion mandate is not approval. No valid approval file is included. See docs/live_execution.md for exact eligibility/approval JSON schema, source binding and commands. Do not fabricate qualifying evidence or copy example equity into a real account.

14. Live orders, cancellation and emergency shutdown

The implementation is disabled by default. This task does not grant live approval. Never create approval/evidence files merely to get past a gate. A current candidate must first pass every independent research, reliability, account and operator review requirement. The current Kraken candidate fails the unchanged drawdown gate and has no prospective qualification evidence, so real-order commands are presently ineligible.

The read-only approval check requires reviewed eligibility and an independently verified API key fingerprint:

```
python -m kraken_quant.live_cli validate-approval --approval "$taskRun/live-
approval.json" --eligibility "$taskRun/live-eligibility.json" --account-fingerprint
<verified-key-fingerprint>
```

The only expected successful output is approval_valid=true and orders_submitted=0. Eligibility binds raw file hashes and canonical candidate identity. Approval binds that eligibility hash, candidate and account fingerprint, an operator, and an active interval of no more than 24 hours. Full schemas are in docs/live_execution.md.

After all separate requirements and approval genuinely exist, these are the implemented bounded interfaces; they are examples of syntax, not authorisation to run them now:

```
python -m kraken_quant.live_cli account-check --database "$taskRun/live.sqlite" --env-
file .env
python -m kraken_quant.live_cli reconcile --database "$taskRun/live.sqlite" --env-file
.env
python -m kraken_quant.live_cli strategy-once --database "$taskRun/live.sqlite" --env-
file .env --history-start 2026-10-01T00:00:00Z --max-orders 5 --day-start-equity
<genuine-recorded-midnight-equity> --approval "$taskRun/live-approval.json"
--eligibility "$taskRun/live-eligibility.json" --enable-live-orders
python -m kraken_quant.live_cli inspect --database "$taskRun/live.sqlite"
python -m kraken_quant.live_cli cancel --database "$taskRun/live.sqlite" --env-file .env
 --order-id <existing-local-order-id> --approval "$taskRun/live-approval.json"
--eligibility "$taskRun/live-eligibility.json" --enable-live-orders
python -m kraken_quant.live_cli watch --database "$taskRun/live.sqlite" --env-file .env
--seconds 60 --approval "$taskRun/live-approval.json" --eligibility "$taskRun/live-
eligibility.json" --enable-live-orders
python -m kraken_quant.live_cli deadman --database "$taskRun/live.sqlite" --env-file
.env --approval "$taskRun/live-approval.json" --eligibility "$taskRun/live-
eligibility.json" --enable-live-orders
```

Substitute only genuine recorded values. Strategy-once uses the sealed factory, sources, parameters and daily schedule, sequentially sells before buys, and stops on an UNKNOWN or unsettled order. Each mutation rechecks approval and risk at the wire boundary. Use one supervised process and database per key. IOC orders, persistent reservations, no ambiguous blind retries, fill deduplication and exchange reconciliation apply. Do not use manual order submission to bypass the frozen runner. Read-only account-check/reconcile must pass and the account must be dedicated and coherent before any separately approved mutation.

Local emergency kill needs no credential and must remain available while the writer holds its process lock:

```
python -m kraken_quant.live_cli kill --database "$taskRun/live.sqlite"
python -m kraken_quant.live_cli inspect --database "$taskRun/live.sqlite"
```

Kill blocks new submissions. It does not promise exchange cancellation. Use the separately authorised cancel interface for known orders or Kraken's trusted UI, then read-only reconcile. The exchange 60-second dead-man timer is armed for execution; supervised watch refreshes it approximately every 20 seconds while healthy. Stop strategy supervision, retain the database and reconcile every acknowledged/UNKNOWN order before resume. Never delete a live DB or release UNKNOWN reservations manually.

15. Troubleshooting and acceptance limits

Stale quotes: inspect exchange timestamps, system UTC, connectivity and market status. Reconnect within bounded limits; never substitute local now, accept crossed books or increase the five-second gate. Metadata must retain its actual observation time. Slow private/dead-man requests are followed by a final pre-send freshness/kill check.

Missing history: supply a complete authenticated/provenance-pinned archive. Kraken REST history retention is limited; do not shorten warm-up silently. Point-in-time expanded-universe work needs genuine historical listing/delisting/liquidity records. Current nine-asset survivor-selected results cannot substitute for those inputs.

Exchange errors: explicit rejection and ambiguous submission are different. Reconcile the former before release; hold UNKNOWN reservations for the latter. Rate limiting is bounded, but writes are never blindly retried. Inspect safe audit codes rather than printing raw signed/private responses.

Database locks: stop competing writers and preserve files. Do not remove locks or copy an active SQLite file manually. Use the online backup interface. Corrupt ledgers, missing known fills or balance changes fail recovery and require investigation; they must not be automatically rebased.

Failed qualification: retain the candidate, data, parameters and failed report. Do not lower gates or tune against inspected OOS. Register a new hypothesis/version for genuinely new research. The current low-correlation historical result remains Sharpe about 0.37 and drawdown about 30.04%, with relative improvement but no established standalone alpha. The separately evaluated Kraken-compatible training candidate also fails drawdown acceptance.

A full software walkthrough is sections 2-9 plus simulation in section 13. It is executed by tests/test_end_to_end_acceptance.py and the parent acceptance commands. Genuine public-feed operation is tested separately. No accelerated replay, simulated exchange or synthetic strategy outcome counts toward future market evidence. Review docs/completion_register.md and the two cumulative logs for exact final test receipts, external dependencies and remaining future observations.